Design Document: Campii - Campus Ride Platform

Project: Campii (Campus Ride Management System)

Deliverable: Design Document (Deliverable 2)

Architecture: MERN Stack (MongoDB, Express.js, React.js, Node.js) with Socket.IO

1. Problem Understanding

Campus environments frequently face internal transportation bottlenecks. Students and staff often struggle to find reliable, short-distance transit between hostels, libraries, and main gates, while student drivers with vehicles lack a streamlined way to offer rides.

The objective is to design and develop a centralized digital platform capable of connecting passengers and drivers in real-time within a campus environment.

Core Challenges Addressed:

1. **Real-Time Availability:** Passengers need to know if drivers are currently active and nearby.
2. **Efficient Dispatching:** Ride requests must not be broadcast blindly. They should be intelligently routed to the nearest available drivers to minimize Wait Time (ETA).
3. **Concurrency & Race Conditions:** Multiple drivers might attempt to accept the same ride simultaneously. The system must ensure a strict 1-to-1 mapping where a ride can only be assigned to a single driver.
4. **State Synchronization:** The "Ride Lifecycle" (Requested → Accepted → Completed / Cancelled) must be perfectly synchronized between the passenger's screen, the driver's screen, and the backend database to prevent "Ghost Rides" (orphaned ride requests).

2. System Architecture

The application follows a modern decoupled **Client-Server Architecture**, utilizing the MERN stack alongside bidirectional WebSockets for real-time features.

2.1 High-Level Architecture

- **Presentation Layer (Frontend):** A responsive Single Page Application (SPA) built with React.js and Vite. It utilizes Tailwind CSS for styling and `react-leaflet` for live map integration.
- **Business Logic Layer (Backend):** A Node.js and Express.js REST API that handles authentication, ride creation, and historical data retrieval.
- **Real-Time Engine:** A Socket.IO server running parallel to the Express app, responsible for maintaining persistent connections with active users, emitting ride requests, and pushing status updates.
- **Data Layer (Database):** MongoDB database utilizing a `2dsphere` index for spatial queries to handle geographic matching.

2.2 Directory Structure Alignment

The system's modularity is reflected in the implemented directory structure:

```
campus-ride-platform/
├── backend/
│   ├── src/
│   │   ├── controllers/      # Business logic (authController.js, rideController
│   │   ├── middlewares/      # JWT verification and error handling
│   │   ├── models/           # Mongoose schemas (User.js, Ride.js)
│   │   ├── routes/           # Express API definitions (authRoutes.js, rideRoute
│   │   ├── sockets/          # Centralized WebSocket event handlers
│   │   ├── utils/            # Helper functions (hashing, formatting)
│   │   └── server.js         # Entry point: Express app & Socket.io initializati
│   └── .env                  # Environment variables
└── frontend/
    ├── src/
    │   ├── assets/           # Static images and icons
    │   ├── components/       # Reusable UI (LiveRideCard.jsx, RideHistoryWidget.
    │   ├── pages/            # View components (Home, Dashboards, Auth pages)
    │   ├── App.jsx           # Main React router wrapper
    │   └── main.jsx          # React DOM mounting point
    └── tailwind.config.js    # UI styling tokens
```

3. Database Schema

The database is built on MongoDB using Mongoose ODM. It relies heavily on geospatial indexing to facilitate the nearest-driver matching algorithm.

3.1 User Collection (`User.js`)

Stores authentication, profile, and real-time tracking data for both Passengers and Drivers.

Field	Type	Description
`_id`	ObjectId	Primary Key
`firstName` / `lastName`	String	User's full name
`email`	String	Unique login identifier
`password`	String	Bcrypt hashed password
`role`	String	Enum: `['passenger', 'driver']`
`vehicle` , `plate`	String	Driver-specific details (Optional for passengers)
`isOnline`	Boolean	Tracks radar status for drivers
`socketId`	String	Current active Socket.IO connection ID
`location`	GeoJSON	`{ type: 'Point', coordinates: [lng, lat] }` (Indexed `2dsphere`)

3.2 Ride Collection (`Ride.js`)

Maintains the state and history of every ride interaction.

Field	Type	Description
`_id`	ObjectId	Primary Key
`passenger`	ObjectId	Reference to `User` collection
`driver`	ObjectId	Reference to `User` collection (Nullable initially)
`pickupLocation`	String	Text description of pickup point
`destination`	String	Text description of drop-off point
`fare`	Number	Calculated or fixed price for the ride
`status`	String	Enum: `['Requested', 'Accepted', 'In Progress', 'Completed', 'Cancelled']`
`createdAt` / `updatedAt`	Timestamp	Automatically managed by Mongoose

4. Entity Relationship Diagram (ERD)

The relationship model strictly enforces that a Ride has exactly one Passenger, and zero-to-one Driver (depending on the lifecycle phase).

5. API Overview

The application utilizes a hybrid communication approach: REST APIs for persistent state changes and WebSockets for ephemeral, high-frequency updates.

5.1 REST API Endpoints (Express.js)

Authentication:

- `POST /api/auth/register` : Registers a new passenger or driver. Returns JWT.
- `POST /api/auth/login` : Authenticates user credentials. Returns JWT & User Object.

Ride Management:

- `POST /api/rides/` : Creates a new ride request and initiates the Socket broadcast.
- `POST /api/rides/accept` : Driver accepts a ride. Implements an atomic lock to prevent dual-acceptance.
- `PUT /api/rides/:id/status` : Updates a ride to `Completed` or `Cancelled` .
- `GET /api/rides/active/:userId/:role` : Fetches the currently active ride for a specific user to persist state across page reloads.
- `GET /api/rides/history/:userId/:role` : Retrieves paginated history and aggregated performance statistics (Total Earnings/Spent, Ride Counts).

5.2 WebSocket Events (Socket.IO)

- `driverOnline` / `driverOffline` : Updates the driver's `isOnline` flag and current `[lng, lat]` coordinates in the DB.
- `passengerOnline` : Registers the passenger's current `socketId` to receive updates.
- `newRideRequest` : Emitted by the server to targeted drivers containing passenger details and pickup locations.
- `rideStatusUpdated` : Emitted by the server to both the specific Passenger and Driver when a ride moves to Accepted, Completed, or Cancelled.

6. Key Design Decisions & Logic

6.1 Geospatial Matching Algorithm

Rather than broadcasting a ride request to every single driver on campus (which wastes bandwidth and frustrates drivers who are too far away), the system implements a proximity-based algorithm.

- **Implementation:** The driver's `location` field is stored as a GeoJSON `Point` and indexed using MongoDB's `2dsphere` .
- **The `$near` Query:** When a passenger requests a ride, the backend runs a `$near` query utilizing the passenger's exact GPS coordinates. The system finds the **nearest 5 drivers** and emits the `newRideRequest` socket strictly to them.
- **Staggered Broadcasting:** If the ride is not accepted within 60 seconds, a recursive `setTimeout` expands the radius by querying the *next* 5 closest drivers (`skip(5)`) until the ride is accepted.

6.2 Atomic Updates & Race Condition Prevention

If two drivers click "Accept Ride" at the exact same millisecond, the system must not assign the ride to both.

- **Implementation:** The `/api/rides/accept` endpoint utilizes Mongoose's `findOneAndUpdate` with a compound query filter: `{ _id: rideId, status: 'Requested' }` .
- **Result:** The database operation is atomic. The first request successfully changes the status to `Accepted` . The second request fails the filter condition (because the status is no longer 'Requested') and returns a `400 Bad Request` , alerting the second driver that the ride was taken.

6.3 The "Ghost Ride" Prevention (State Cleanup)

During early testing, rapid network disconnects or impatient clicking could result in orphaned "Requested" rides persisting in the database, locking up the frontend UI (Ghost Rides).

- **Implementation:** A "Ghost Buster" mechanism was introduced in `rideController.js`.
- Before a passenger is allowed to create a new ride, the system issues an `updateMany` command to automatically change any of their existing `Requested` rides to `Cancelled`. This guarantees a 100% clean state and ensures a passenger can only have one active request propagating through the WebSocket network at a time.

6.4 Component-Driven UI & Map Integration

- The frontend strictly adheres to React component reusability. Complex UI elements like `LiveRideCard.jsx` and `RideHistoryWidget.jsx` are abstracted into their own files.
- **Mapping:** `react-leaflet` was chosen over Google Maps to keep the application lightweight, free of API quota restrictions during development, and easily customizable for small geographic areas like a university campus.

7. Conclusion

The Campii platform successfully fulfills the Problem Statement by delivering a robust, real-time campus transit solution. By combining the persistent data reliability of REST APIs with the instantaneous communication of WebSockets, the system maintains strict data consistency across all user types. Furthermore, the inclusion of MongoDB spatial indexing positions the platform for excellent scalability, ensuring that as the user base grows, ride dispatching remains highly localized and efficient.